Immune Deep Reinforcement Learning based Path Planning for Mobile Robot in Unknown Environment

Chengliang Yan[1], Guangzhu Chen [2,*], Yang Li[1], Fuchun Sun[3],Yuanyuan Wu [2]

[1] *College of mechanical and electrical engineering, Chengdu University of Technology, Chengdu, China*

2 College of Computer Science and Cyber Security ,Chengdu University of Technology ,Chengdu,China

3 School of Mechanical Engineering, Cheng Du University, Chengdu,China

Corresponding author, Guangzhu Chen, E-mail: cgzhu@126.com

Abstract: An upgraded DDPG integrating kinematics analysis and immune optimization (KAI-DDPG) is presented to address the current drawbacks of DDPG in path planning. To begin, the orientation angle reward component, linear velocity reward factor, and safety performance reward factor are added into the DDPG reward function based on kinematic modeling and analysis of mobile robots. A multi-objective performance index is also presented to turn the path planning problem into a multi-objective optimization problem. The KA-DDPG is proposed, which uses the orientation angle, linear speed, and safety degree as evaluation indexes, and the information entropy to alter the influence coefficient of the multi-objective function in the reward function. Then, in order to address KA-low DDPG's learning and training efficiency, KAI-DDPG is proposed, which uses immune optimization to optimize the experience samples in the experience buffer pool of KA-DDPG. Finally, using the gazebo simulation platform, the simulation environment is established, and the performance indices of traditional path planning and the suggested techniques are compared. The findings suggest that KAI-DDPG can mitigate DDPG's drawbacks, such as a protracted training cycle and a poor path planning technique, while also having greater application.

Keywords: mobile robot; path planning; DDPG; reward function; immune algorithm

1 Introduction

With the progress of science and technology and the development of society, mobile robots have been widely used in medical, entertainment, service, security, industry, and so on. Autonomous navigation is the main ability to ensure that the robot does not collide with obstacles in the process of reaching the target point, which has an important influence on the performance of mobile robots. At present, the mainstream autonomous navigation method is task-based navigation based on a map. The process is divided into three parts: location, mapping, and path planning. The location mostly adopts Adaptive Monte Carlo Localization (AMCL) [1], while the mapping mainly adopts Kalman filter slam, graph optimization slam [2], etc. Path planning is divided into global path planning and local path planning, and the global path planning generally adopts the A* algorithm [3] and the RRT algorithm [4]. The dynamic window method (DWA) [5] and the TEB algorithm [6] are generally used in local path planning. The path planning method based on the map needs a lot of manpower, material, and financial resources during the process of building the map, and it is difficult to adapt to the dynamic and large-scale environment. As a result, as the need arises, the map-less path planning method emerges. There is no need to build a map for map-less path planning. The robot realizes path planning by interacting with the external environment and exploring independently.

In map-less path planning, the autonomous learning ability of a mobile robot is the key ability to measure the robot's autonomous path exploration in an unknown environment. In recent years, the reactive map-less path planning method based on reinforcement learning has been given more and more attention because it has the advantages of no supervision, no prior knowledge, and strong autonomous learning ability. The development of reinforcement learning can be traced back to the 1950s. In 1954, Minsky first proposed the concept of reinforcement learning [7]. In 1957, Bellman

Electronic copy available at: https://ssrn.com/abstract=4070518

proposed the dynamic programming of a stochastic discrete Markov decision process for solving optimal control problems [8], which simplified the solution process of reinforcement learning. In 1989, Watkins proposed the Q-learning method [9], which further expanded the research direction of reinforcement learning.

But after that, with the emergence of a large number of other excellent machine learning algorithms, reinforcement learning once again fell to the bottom because of a lack of breakthrough progress. Until 2013, the Deepmind team combined reinforcement learning with deep neural networks, proposed the Deep Q network (DQN), and successfully applied it to the Atari game, making the research of reinforcement learning once again full of vitality. In 2015-2016, Alphago, developed by the team, defeated the world's top professional chess players and attracted a large number of domestic and foreign scholars to study deep reinforcement learning.

On the other hand, deep reinforcement learning can make agents adapt to many complex problems and has a certain universality. At present, path planning methods based on deep reinforcement learning have gradually become the focus of research, such as DQN [10], double DQN [11], dueling DQN [12] and other deep reinforcement learning algorithms, but these algorithms can only transform high-dimensional continuous actions into discrete action sets for processing. With this transformation process, the structure of the action domain will be changed. However, the structure of the action domain is very important when completing path planning tasks [13]; Lillicrap et al. [14] combined DPG (deterministic policy gradient) algorithm [15] with DQN, DDPG (deep deterministic policy gradient) algorithm is proposed.DDPG can be directly used to solve the continuous action space problem with a large spatial dimension. In recent years, a large number of articles and experiments have proved that DDPG not only performs stably in continuous action space tasks, but also takes much less time to find the optimal solution than DQN.

At present, there are two shortcomings in the DDPG-based path planning problems of mobile robots:

- The mobile robot based on DDPG path planning can’t obtain the optimal motion strategy, which leads to the trajectory drift of the mobile robot. Because the setting of the DDPG reward function does not combine with the motion characteristics of the mobile robot,
- It can’t make use of the excellent motion strategy experience in the experience buffer pool, resulting in the low learning and training efficiency of the algorithm, because the sampling mechanism used in DDPG for the experience buffer pool is random and evenly distributed sampling.

Aiming at the first problem, Deng et al. [16] proposed a new multi-objective vehicle following decision algorithm based on the modular design of the DDPG reward function according to the longitudinal kinematic characteristics of the following vehicle and the pilot vehicle. Han Jinliang et al. [17] proposed a flexible AC algorithm based on a multi-layer parallel attention mechanism to optimize the motion strategy of mobile robots. According to human driving data, Zhu Meixin et al. [18] combined driving safety, efficiency, and comfort, designed a DDPG reward function, and proposed a speed control model for vehicle following based on reinforcement learning. On the basis of this research, this paper designs the reward function of DDPG, combined with kinematics analysis, and obtains a better motion strategy, so as to obtain an excellent motion trajectory. For the second problem, Zhang et al. [19] proposed a DDPG based on failure experience correction and applied the algorithm to automatic driving. On the basis of the above methods, further optimization was made in the follow-up research. Zhang et al. [20] applied the priority experience playback method to DDPG to improve the sampling efficiency. At present, TDC-DDPG and RC-DDPG are two improved algorithms for DDPG [21]. These two improved algorithms classify the experience groups in the experience buffer pool according to the average value of td-error and the average value of immediate reward, respectively, and then use the experience according to the priority. In this paper, the immune algorithm is used to optimize the excellent experience group in the experience buffer pool, which speeds up the convergence speed of DDPG.

The main contributions to this paper are as follows:

(1) Combined with the kinematics analysis of the mobile robot, the orientation angle reward factor, linear velocity reward factor, and safety performance reward factor are introduced into the DDPG reward function to make the mobile robot plan a smoother trajectory, shorten the navigation distance and time, and improve the navigation success rate;

(2) The information entropy is used to adjust the influence coefficient in the reward function, which makes the design of the reward function more reasonable, adapts to the simple, complex, and

Electronic copy available at: https://ssrn.com/abstract=4070518

dynamic environment, and enhances the generalization of the algorithm;

(3) In order to satisfy the requirements of experience diversity, the immune algorithm is used to deeply mine the excellent experience in the experience buffer pool, so that the mobile robot can obtain a better motion control strategy and improve the utilization rate of experience samples, so that the reward value can reach a higher stable value faster and the convergence speed of DDPG is accelerated.

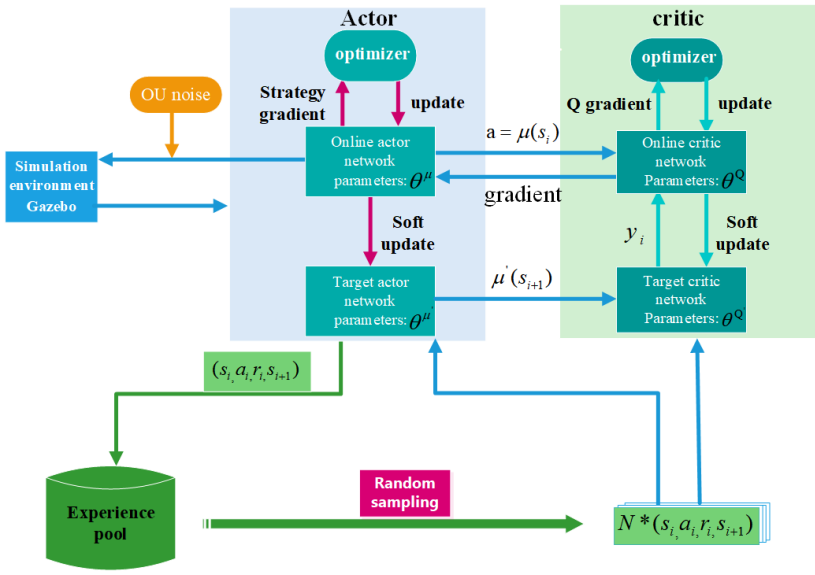


Figure 1 DDPG general framework

2. Design of DDPG reward function based on kinematics analysis

2.1 Brief introduction of DDPG

DDPG introduces an actor critical mechanism and uses two independent networks to approximate the actor and critic, respectively [14]. The actor network and the critic network both use fully connected neural networks with two hidden layers. The four network structures of DDPG are shown in Figure 1. Two of them are the online actor network and the online critic network, in which the mobile robot interacts with the environment in real time. In the online actor network, the state s is taken as the input, the actor network $\mu(s|\theta^{\mu})$ is estimated by the weight parameter θ^{μ}, and the action a is output. In an online critic network, state s and action a are used as inputs, and a weight parameter θ^{Q} is used to estimate the critic network $Q(s,a|\theta^{Q})$ and its output value. The other two networks are the target actor network μ' and the target critic network Q', which are used to ensure that the online critic network Q is more stable and easier to converge in the learning process.

The weights of the target actor network $\mu'(s_{i+1}|\theta^{\mu'})$ and target critic network $Q'(s_{i+1},\mu'(s_{i+1}|\theta^{\mu'})|\theta^{\theta'})$ are obtained from the online actor network and online critic network by the soft update method, as shown in formula 1 and formula 2:

$$\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'} \tag{1}$$

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'} \tag{2}$$

Where $\theta^{\mu'}$ is the target actor network's weight parameter, τ is the soft update's efficiency, and $\theta^{Q'}$ is the target critic network's weight parameter.

The ultimate goal of DDPG is to output the optimal action a by optimizing the parameter θ^{μ} when the current state s is entered in the online actor network. In the DDPG, θ^{Q} is used to update the formula 3:

$$\nabla_{\theta^{\mu}} J \approx \frac{1}{N}\sum_{i} \nabla_{a} Q(s,a|\theta^{Q})\,|_{s=s_i,a=\mu(s_i)} \cdot \nabla_{\theta^{\mu}}\mu(s|\theta^{\mu})|_{s_i} \tag{3}$$

Electronic copy available at: https://ssrn.com/abstract=4070518

Where N is the number of experience samples and J is the gradient of the online actor network θ^{μ}.

The quality of the online actor network output is determined by the online critic network, and the θ^{Q} in the online critic network is updated by using the minimum loss function; To make the Q value and y_i value closer in the critic network, the setting of online network loss function L is shown in formula 4:

$$L = \frac{1}{N}\sum\left(y_i - Q(s_i, a_i; \theta^{Q})\right)^2 \tag{4}$$

Where y_i is composed of the reward value r_i and the target Q value in the target critic network, the reward value r_i is calculated by the artificial reward function, and as shown in formula 5:

$$y_i = r_i + \gamma Q'\left(s_{i+1}, \mu'\left(s_{i+1}; \theta^{\mu'}\right); \theta^{Q'}\right) \tag{5}$$

Where γ is the learning rate, which can dynamically adjust the update speed of the network.

Because the gradient update of θ^{μ} in online actor network is affected by the gradient update of θ^{Q} in online critic network, the overall quality of parameter update of θ^{Q} in online critic network indirectly affects the output action of actor network. To sum up, the quality of motion strategy in actor network depends on the setting rules of reward function in online critic network.

2.2 Path planning framework based on DDPG

As shown in Figure 2, the input of DDPG is the characteristic information of the surrounding environment obtained by the mobile robot through the sensor, that is, the average distance information $d(d1, d2, \ldots, d10)$ of 10 laser beams distributed in front of the robot measured by the lidar, the current robot deflection angle θ obtained from odometer information, the orientation angle $\theta_{orientation}$ relative to the target point and the distance D from the target point, etc. Through DDPG, the motion strategy of the mobile robot is output, that is, the linear velocity value v and the angular velocity value ω.

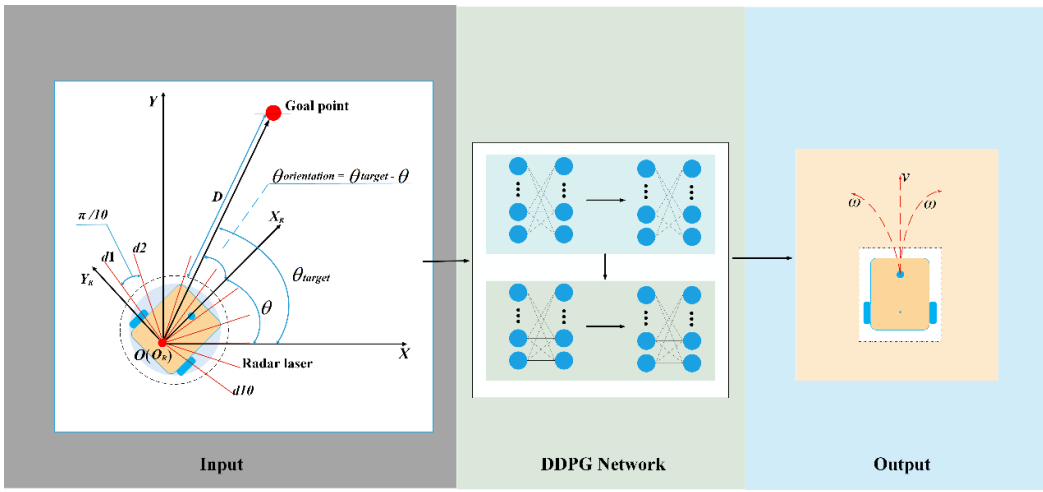


Figure 2 Path planning framework based on DDPG

2.3 Kinematics analysis of mobile robot

The motion characteristics of mobile robots will greatly affect the effect of path planning. In order to obtain a better and more reliable path planning method, KA-DDPG (DDPG based on kinematic analysis) is proposed when DDPG is applied to the path planning of mobile robots. This algorithm combines the kinematic analysis of mobile robots with information entropy theory to set the reward function. In order to verify the rationality of the reward function setting, a two-wheel drive differential mobile robot model with encoder, inertial measurement unit and lidar is built in the ROS/Gazebo simulation environment, as shown in Figure 3, and its relevant parameters are shown in Table 1. The information from the encoder and inertial measurement unit is fused by the Kalman filter [22] algorithm to obtain odometer information, including position information and

Electronic copy available at: https://ssrn.com/abstract=4070518

quaternion information. As shown in Figure 2, the attitude angle θ of the mobile robot can be obtained from the quaternion information and θ_{target} can be obtained from the coordinate information of the car and the target point. $(\theta_{target} - \theta)$ is the orientation angle of the mobile robot relative to the target point $\theta_{orientation}$.

Table1 Main parameters of Mobile robot

Parameter	value
Robot size(mm)	306×282.57×142.35
Max angle speed(rad/s)	1.82
Max speed(m/s)	0.26
Max payload(kg)	30

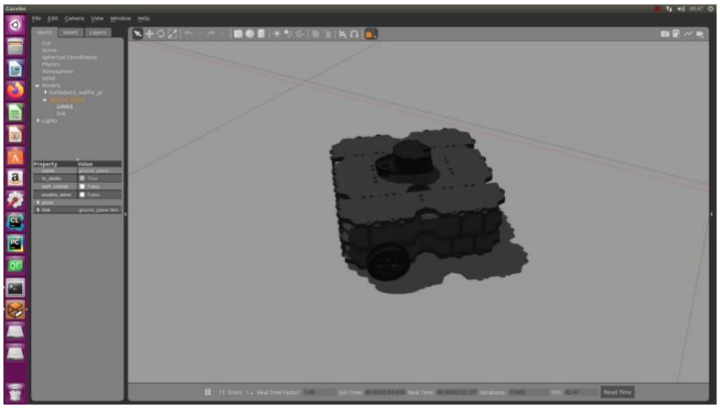


Figure 3 Model of the two-wheel drive differential robot in gazebo

The general kinematics formula of mobile robot system is shown in formula 6:

$$S(t+1) = f\big(S(t), u(t)\big) + \delta_t \tag{6}$$

Where $S(t)$ is the current state of the system, $S(t+1)$ is the state of the next sampling period, $u(t)$ is the control variable in the system, δ_t is the amount of interference in the system.

The kinematic model of the two-wheel drive differential mobile robot is shown in Figure 4, and its mathematical model is shown in formula 7.

$$S(t+1) = \begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix} = f\left(\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}, u(t)\right) + \delta_t$$
$$= \begin{bmatrix} vTcos\left(\theta_t + \frac{\omega T}{2}\right) \\ vT\,sin\left(\theta_t + \frac{\omega T}{2}\right) \\ \omega T \end{bmatrix} + \delta_t \tag{7}$$

Where T is the unit sampling period, x_t is the x axis coordinate of the current state, y_t is the y axis coordinate of the current state, θ_t is the rotation angle of the current state, x_{t+1} is the x axis coordinate of the next state, y_{t+1} is the y axis coordinate of the next state, θ_{t+1} is the rotation angle at $t+1$, $u(t) = (vT, \omega T)$ is the control variable of the mobile robot, v is the linear velocity of the mobile robot, ω represents the angular velocity, θ_t is the rotation angle at time t, δ_t is represented by Gaussian noise in the navigation algorithm code.

Because of the limited torque of its own motor, when the maximum linear velocity of the mobile robot is v_{max}, the maximum acceleration a_{max} is considered. It takes time to stop, so the maximum safe distance X_{max} is set, as follows:

$$t_{max} = \frac{v_{max}}{a_{max}} \tag{8}$$

$$X_{max} = v_{max}t_{max} - \frac{1}{2}a_{max}{t_{max}}^2 \tag{9}$$

When the mobile robot turns, the left wheel speed v_L of the mobile robot is considered and right wheel speed v_R is different, half of the distance between the left and right wheels is L_R. In

Electronic copy available at: https://ssrn.com/abstract=4070518

this case, the turning radius of instantaneous curvature R_c can be obtained. As shown in formula 10:

$$R_c = \frac{L_R(v_R + v_L)}{v_R - v_L} \tag{10}$$

In conclusion, when the speed of one of the wheels is 0, the mobile robot rotates. According to formula 10, the minimum instantaneous radius of curvature L_R can be obtained, the value of R_c. The turning expansion distance L_s set by the mobile robot. As shown in formula 11, it can help the mobile robot reduce the collision probability.

$$L_s = L_R - (R - L_e) \tag{11}$$

Where R is the radius of the circumscribed circle of the robot's physical dimension, L_e is the deviation distance between the geometric center and the turning center of the mobile robot.

From the turning expansion distance and the maximum safety distance, the final expansion distance L can be obtained, as shown in formula 12.

$$L = max(X_{max}, L_s) \tag{12}$$

Then the safe radius of the mobile robot is r_s, It can be obtained from formula 13.

$$r_s = L + R \tag{13}$$

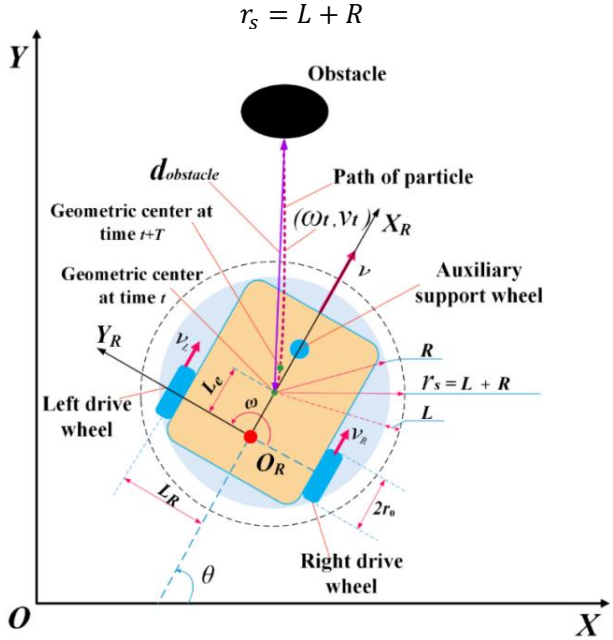


Figure 4 Kinematic model of mobile robot

2.4 Set reward function

As shown in formula 14, the linear combination of features is used to express the reward function $r(s,a)$[23-24].

$$r(s,a) = \theta^T \varphi(s,a) \tag{14}$$

Where θ^T is the weight vector, that is, the weight of reward factors for each part of the reward function; $\varphi(s,a)$ is the feature vector of a single feature point in the space (state-action), that is, the reward function corresponding to each state.

According to the requirements of path planning and the results of kinematics analysis of mobile robots, the reward factor of orientation angle $r_{orientation}$ is calculated, the safety performance reward factor r_{safe} and linear velocity reward factor $r_{velocity}$ are the reward factors of reward function. In addition, two dynamic reward factors $r_{collide}$ and r_{goal} are set, $r_{collide}$ measures the collision between the mobile robot and an obstacle, r_{goal} measures whether the mobile robot reaches the target successfully. The above five reward factors consider the internal and external factors of the robot, which is more in line with the actual situation of navigation, so as to improve the quality and efficiency of DDPG path planning, and enhance the environmental adaptability.

The reward function Re is composed of the above five reward factors, and its expression is

Electronic copy available at: https://ssrn.com/abstract=4070518

shown in formula 15.

$$Re = \sigma(\alpha r_{orientation} + \beta(r_{safe} + r_{velocity})) + r_{collide} + r_{goal} \tag{15}$$

In order to avoid the influence of some reward factors on the value of the reward function in some specific circumstances, we set σ as the scaling factor of $r_{orientation}, r_{safe}$ and $r_{velocity}$ in each complete navigation task that can be adjusted dynamically according to the experimental results.

The expression of $r_{orientation}$ is shown in formula 16.

$$r_{orientation} = \begin{cases} \left(\frac{\pi}{2} - \theta_{orientation}\right), if \quad 0 < \theta_{orientation} < \pi \\ \left(\theta_{orientation} - \frac{3\pi}{2}\right), if \quad \pi < \theta_{orientation} < 2\pi \end{cases} \tag{16}$$

It can be seen from formula 16 that the smaller the orientation angle $\theta_{orientation}$ is, the greater the value of the reward factor will be.

r_{safe} is the reward factor to judge the distance between the mobile robot and the obstacle on its trajectory, The calculation process of r_{safe} is shown in formula 17.

$$r_{safe} = d_{obstacle} - r_s \tag{17}$$

Where r_s is the safe radius of the mobile robot, which can be determined by formula 13; $d_{obstacle}$ is the distance between the geometric center of the mobile robot and the obstacle, which can be calculated by formula 18. It can be seen from the above formula that within the determination range set according to different navigation environments, the larger the $d_{obstacle}$ is, the greater the value of reward factor will be.

$$d_{obstacle} = \sqrt{(x_{current} - x_{obstacle})^2 + (y_{current} - y_{obstacle})^2} \tag{18}$$

Where $(x_{current}, y_{current})$ is the world coordinate of the geometric center of the mobile robot, which can be determined by coordinate transformation, odometer information and the structure size of the mobile robot. $(x_{obstacle}, y_{obstacle})$ is the coordinates of obstacles in the current world coordinate system, which can be determined by odometer information and lidar information.

$r_{velocity}$ is used to evaluate the forward speed of the mobile robot on the corresponding trajectory. In the unit sampling period, the higher the speed is, the greater the value of the reward factor is. The calculation process is shown in formula 19.

$$r_{velocity} = v_{current} \tag{19}$$

Where $v_{current}$ is the linear velocity of the mobile robot.

These three reward factors are important parts of the reward function, and they are indispensable. If only r_{safe} and $r_{velocity}$ is maximized, the mobile robot will always move in the barrier free space and will not move to the target position. With a single maximization $r_{orientation}$, the mobile robot will be unable to avoid the first obstacle in its forward direction. By combining the above three reward factors, the mobile robot can quickly avoid the obstacles and move towards the target points.

Considering $r_{orientation}$, r_{safe} and $r_{velocity}$ have great influence on path planning strategy, in order to make the value of these three reward factors more reasonable, α and β are introduced as the influence factors of $r_{orientation}$, r_{safe} and $r_{velocity}$. In order to evaluate the uncertainty of α and β factors, information entropy is introduced. Information entropy is a measure of the amount of information needed to eliminate uncertainty. The more information is introduced to eliminate uncertainty, the higher the information entropy is, and vice versa. The calculation formula for information entropy $H(X)$ is shown in formula 20.

$$H(X) = -\sum_{i=1}^{2} p_i \log_2(p_i) \tag{20}$$

Where X is an unknown event, p_i is the probability of an X event.

The last collision (or reaching the target point) of mobile robot to the next collision (or reaching the target point) is regarded as a round. The $r_{orientation}$ of each round of reaching the target point is calculated every 10000 time steps. Information entropy $H(orientation)$ and information entropy $H(sv)$ of the sum of $r_{velocity}$ and r_{safe}. As shown in formulas 21 and 22.

$$H(orientation) = -p_{orientation}^{good} \log_2\left(p_{orientation}^{good}\right) - p_{orientation}^{bad} \log_2\left(p_{orientation}^{bad}\right) \tag{21}$$

Electronic copy available at: https://ssrn.com/abstract=4070518

$$H(safe) = -p_{sv}^{good}\log_2\left(p_{sv}^{good}\right) - p_{sv}^{bad}\log_2(p_{sv}^{bad}) \tag{22}$$

Where The probability that $r_{orientation}$ is positive is $p_{orientation}^{good}$ is the probability of a negative value, denoted as $p_{orientation}^{bad}$; The probability that $r_{safe} + r_{velocity}$ is positive is p_{sv}^{good} is the probability of a negative value, denoted as p_{sv}^{bad}.

The information entropy method is used to judge the value of α and β, as follows:

$$\alpha = \frac{H(orientation)}{H(orientation) + H(sv)} \tag{23}$$

$$\beta = 1 - \alpha \tag{24}$$

3 KAI-DDPG

3.1 Problem description

The KA-DDPG proposed in the previous section extends the soft update mechanism of the traditional DDPG, which can make the target network convergence faster, but also make the training time longer.

KA-DDPG also uses the experience playback mechanism of traditional DDPG. The so-called experience playback mechanism stores the experience sample data obtained from the interaction between agent and environment in the experience buffer pool at each time step. When network training is needed, a small batch of data is randomly extracted from the experience buffer pool for training. By introducing the experience playback mechanism, the time correlation between samples can be eliminated and the network convergence speed can be accelerated. However, KA-DDPG generally only extracts experience samples, and rarely processes experience samples, so the utilization rate of data is not high, which leads to the slow speed of the agent in obtaining the optimal strategy.

Through the above analysis, we can know that the experience buffer pool stores a large number of experience samples, which is an important part of KA-DDPG. If we make some reasonable optimizations to the experience buffer pool, it will have a huge impact on the performance of KA-DDPG. Through investigation, it has been concluded that the immune algorithm [25] is a new intelligent search algorithm inspired by the biological immune system. It is a heuristic random search algorithm combining deterministic and random selection, and has the ability of "exploration" and "mining". It has the advantages of global search ability, diversity maintenance mechanism, strong robustness, parallel distributed search mechanism, etc. This paper introduces immune algorithm to optimize the experience samples in the KA-DDPG experience buffer pool, and proposes KAI-DDPG (KA-DDPG based on Immune Optimization). This algorithm solves the problems of long training time and slow speed of obtaining the optimal strategy for KA-DDPG.

3.2 Summary of KAI-DDPG

When training the KA-DDPG network, some experience samples are randomly and evenly distributed from the experience buffer pool as training samples, in order to eliminate the high correlation of experience samples over time. The experience buffer pool is used to store the experience samples (s_t, a_t, r_t, s_{t+1}) generated by the interaction between the mobile robot and the environment, where s_t is the state entered at a time t, a_t is the action executed at a time t, s_{t+1} is in s_t state to execute a_t After t, the state of $t+1$, r_t is the reward value at time t, which represents the evaluation of the action in the whole navigation task in the current experience group, namely the larger r_t is, the better the action strategy of the experience group is, and the better the path planning strategy is, so raising the value of r_t is regarded as the problem to be optimized.

Although the sampling mechanism of KA-DDPG can eliminate the time correlation and improve the network training effect, if the samples are directly extracted from the experience buffer pool without data processing, the data cannot be used efficiently, and when there are many criteria for the reward function, the speed of obtaining the optimal navigation strategy will slow down. Therefore, in this paper, we use immune algorithm to evaluate the r_t of the experience group in the KA-DDPG experience buffer pool. The overall framework of KAI-DDPG is shown in Figure 5.

Electronic copy available at: https://ssrn.com/abstract=4070518

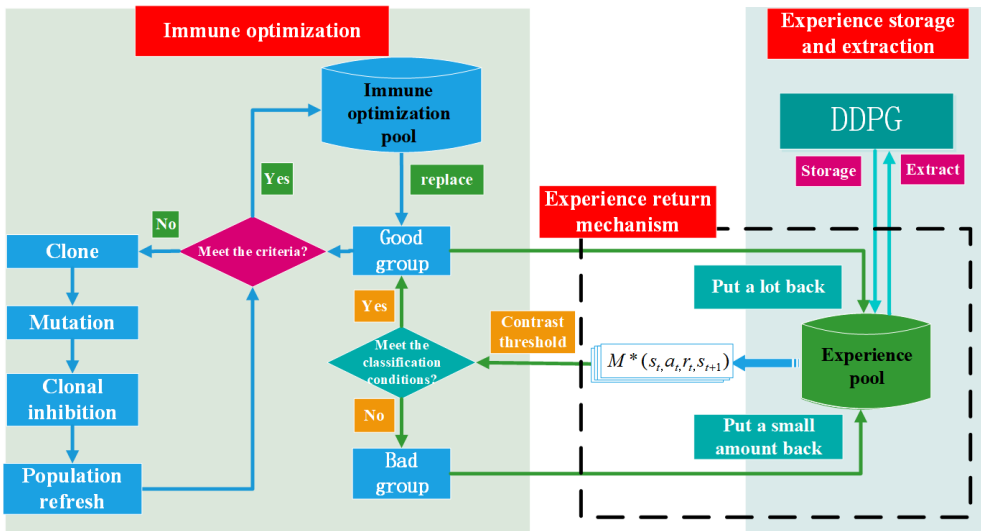


Figure 5 KAI-DDPG frame diagram

3.3 KAI-DDPG specific analysis

First, M groups of experience samples are extracted from the experience buffer pool, and then the average reward value $r_{average}$ is used as the classification standard to classify the M group of experience samples. As shown in formula 24, the experience group with a higher average reward value is put into the Good group, and the experience group with a lower average reward value is put into the Bad group.

$$r_{average} = \frac{\sum_{i}^{i+M-1} r_i}{M}, i = 1, M+1, 2M+1, \ldots \tag{24}$$

In the Good group, r_t is regarded as the initial population to be optimized, and the size of the population is $M/2$. The average reward value of each group of experience samples in the Good group is r_{ga} is regarded as the evaluation criteria of r_t, as shown in formula 25, the experience samples that meet the evaluation criteria are added to the immune optimization pool, and the experience samples that do not meet the evaluation criteria are used for immune operation.

$$r_{ga} = \frac{2\sum_{g}^{g+\frac{M}{2}-1} r_g}{M}, g = 1, \frac{\mathrm{M}}{2}+1, \mathrm{M}+1, \ldots \tag{25}$$

Where g is the serial number of each experience sample in the Good group.

The process of immunization is as follows.

a)Immune clone operation

Clone 10 groups of experience samples that did not meet the evaluation criteria in Good group, and record them as the clone group.

b)Immune mutation operation

According to the reward function Re, the reward value r_g in the clone group is analyzed respectively, and the size of $r_{orientation}$,r_{safe} and $r_{velocity}$ is obtained, if $r_{orientation}$ is negative, the diagonal velocity in a_g is used for mutation operation; If $r_{velocity}$ is too small, The linear velocity in a_g is used for mutation operation. If r_{safe} is negative, the diagonal velocity and linear velocity are mutated. Where, the variation operation conforms to the standard normal distribution, as shown in formula 26 and formula 27, where μ= 0, σ = 1.

$$a_g \sim N(\mu, \sigma^2) \tag{26}$$

$$a_g{}' = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(a_g-\mu)^2}{2\sigma^2}\right) \tag{27}$$

c)Clonal inhibition

According to the mutated $a_g{}'$ recalculate to get $r_g{}'$ A new experience group $(s_g, a_g{}', r_g{}', s_{g+1})$ was formed, calculate and compare the average reward value of the cloned group and the clone group, and keep the experience group with the highest average reward value.

Electronic copy available at: https://ssrn.com/abstract=4070518

d)Population refresh operation

In the experience group after the clone suppression operation, the experience sample with the higher reward value is used to replace the experience sample with the lower reward value in the initial population, and the new species group is obtained. Next, according to the evaluation criteria, judge whether the new population is put into the immune optimization pool or continues the immune operation. Finally, the experience samples in the Good group are replaced by the experience samples in the immune optimization pool.

Through the above immune operation, r_t of the experience group in the Good group has been significantly improved. Finally, the experience group of Good group and Bad group is put back into the experience buffer pool for DDPG training. It is worth noting that through the observation of the effects of the following simulation experiments, this paper further optimizes the experience putting back mechanism, that is, to extract a large number of experience groups from the good group and put them back, which can speed up the training process; at the same time, to extract a small number of experience groups from the bad group and put them back, which can avoid the problem of over fitting and local optimization of DDPG. To sum up, the pseudo code of KAI-DDPG is as follows.

Algorithm KAI-DDPG flow

initialization:

The weight parameters θ^Q and θ^μ of critic network $Q(s,a|\theta^Q)$ and actor network $\mu(s|\theta^\mu)$ are initialized randomly

Initialize the target networks $Q'$and μ', where the network weight parameters are $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize experience pool $\mathcal{R}$

Repeat experience trajectory (episode):

Randomly initialize process $\mathcal{N}$ for action exploration

The initial state value s_0 is obtained

Repeat t to T (step):

(1) According to the current strategy with noise, calculate the action a of the current time step $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$

(2) Perform action a_t. And record the reward r_t and the new state s_{t+1}

(3) Storage conversion experience (s_t, a_t, r_t, s_{t+1}) in the experience pool $\mathcal{R}$

If (t % $M == 0$ and $t > 0$):

M groups of experience samples (s_i, a_i, r_i, s_{i+1}) are randomly sampled from the experience buffer pool.

According to $r_{average}$ put M group of experience samples into Good group and Bad group respectively.

Clone the experience samples that did not meet the criteria in the Good group, and get the clone group.

The clone group was mutated and inhibited to generate new samples $(s_i, a_i', r_i', s_{i+1})$, and put into the immune optimization pool.

Replace the experience samples in the Good group with the experience samples in the immune optimization pool.

Put the experience samples in the Good group and Bad group back to the experience cache pool $\mathcal{R}$.

(4) A small batch of N groups of conversion experience samples (s_i, a_i, r_i, s_{i+1}) are randomly sampled from the experience buffer pool $\mathcal{R}$.

Setting:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$$

(5) Update critic network with minimum loss function:

$$L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$$

(6) The gradient strategy algorithm is used to update the actor network:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i (\nabla_a Q(s,a|\theta^Q)\,|_{s=s_i, a=\mu(s_i)} \cdot \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s=s_i})$$

(7) Update target network:

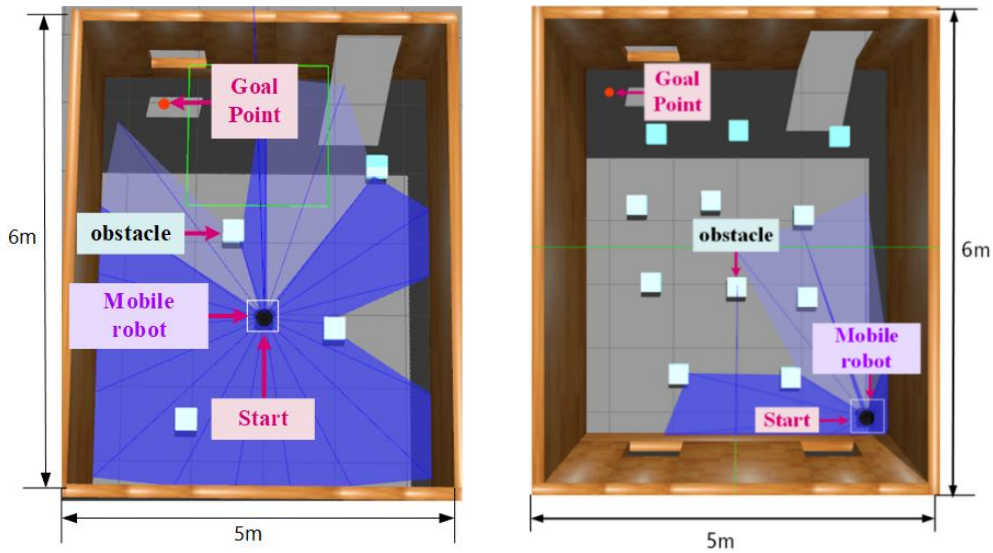
Electronic copy available at: https://ssrn.com/abstract=4070518

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'}$$

4. Experimental data and analysis

To test the efficacy of KAI-DDPG, an experimental platform with Intel i7-8700CPU and NVIDIA Geforce GTX 1070Ti as hardware and ubuntu16.04 and tensorflow1.8.0 as software is built. The relevant experimental parameters are shown in Table 2. As shown in Figure 6, the navigation simulation scene of a mobile robot (simulating the real 5m × 6m indoor environment) is built in ROS/Gazebo, and the dynamic and kinematic models of the mobile robot are established. A navigation simulation scene includes a simple simulation scene, a dynamic simulation scene and a complex simulation scene. Simple simulation scene is indoor environment with few obstacles, dynamic simulation scene is indoor environment with dynamic changes of obstacles, and complex simulation scene is indoor environment with dense obstacles.

The simulation experiments include simple simulation scenes, dynamic simulation scenes, navigation with a map based on A* algorithm, map-less navigation based on DWA algorithm, and map-less navigation based on DQN, DDPG, KA-DDPG and KAI-DDPG in complex simulation scenes. In navigation with map, the map used is a grid map constructed by the SLAM algorithm, which can be displayed in the Rviz visual interface. The grid map content includes the coordinate information of all positions on the map and the characteristic information of obstacles. In map-less navigation, the obstacle information can't be predicted and the navigation path can't be planned in advance.



(a) simple simulation scene (b) complex simulation scene

Figure 6 Mobile robot and obstacle in simulation environment

Electronic copy available at: https://ssrn.com/abstract=4070518

Table 2 Hyper parameters

Parameter	value
Sampling batch size	128
Experience pool size	100000
Discount factor	0.99
Learning rate	0.0001
Network update frequency	0.001
Action dimension	2
State dimension	16
Maximum scanning distance (m)	3.5
Immunization batch	1000
Iterative immune times	500

4.1 Navigation experiment in simple simulation scene

4.1.1 Comparison with other navigation methods

In the simple simulation scene, the navigation trajectories of navigation with map method based on A* algorithm, map-less navigation method based on DWA algorithm and map-less navigation method based on deep reinforcement learning are compared respectively. The navigation trajectories of various navigation methods are shown in Figure 7. It can be seen from the Figure that the navigation trajectories based on KA-DDPG are shorter and smoother than those based on DDPG, DQN and DWA algorithm; moreover, compared with the navigation with map based on A* algorithm, the navigation trajectories based on KA-DDPG are safer, and the turning angle is more in line with the motion characteristics of mobile robot.

In addition, this experiment records the time required to successfully complete the navigation using each depth reinforcement learning navigation method to prove the effectiveness of KA-DDPG, as shown in Table 3.

Table 3 Navigation time of deep reinforcement learning navigation method

Navigation algorithm	Time consuming(s)
DQN	35
DDPG	31
KA-DDPG	24

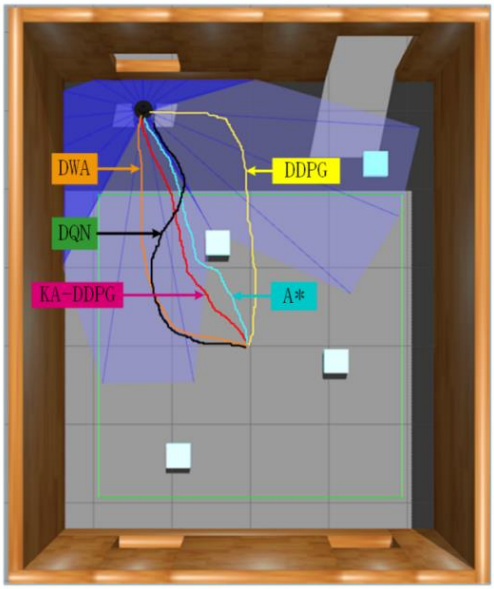


Figure 7 Trajectory of mobile robot based on navigation algorithms in simple simulation scene

Electronic copy available at: https://ssrn.com/abstract=4070518

4.1.2 The success rate of map-less navigation based on DDPG and KA-DDPG

In the training process, the curve of navigation success rate with time step based on DDPG and KA-DDPG is shown in Figure 8. By comparing the navigation success rate curves of DDPG and KA-DDPG, the calculation shows that DDPG tends to converge in about 210000 time steps, and the average navigation success rate after this is about 89.14%. KA-DDPG tends to converge in about 180000 time steps. After that, the average navigation success rate is about 94.93%, which is 5.79% higher than DDPG.

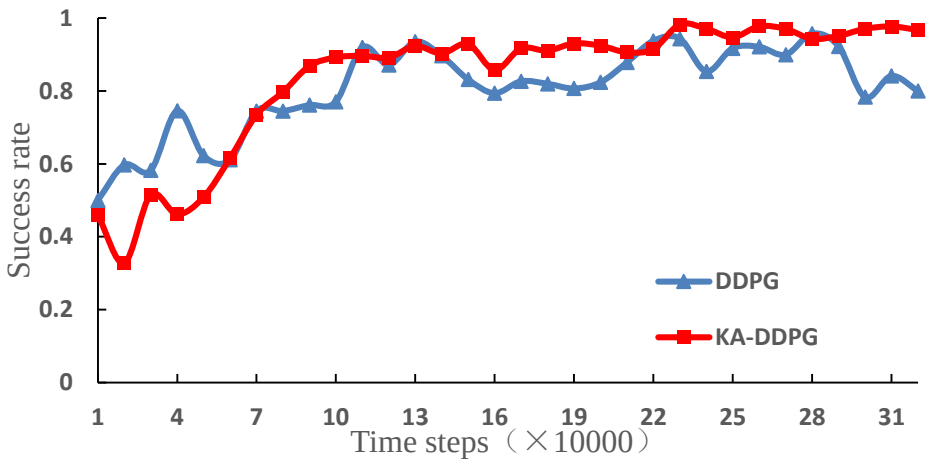


Figure 8 Navigation success rate curves of DDPG and KA-DDPG in simple simulation scene

4.1.3 Training results of map-less navigation based on KA-DDPG and KAI-DDPG

In the training process, the average reward value of KA-DDPG and KAI-DDPG changes with each time step, as shown in Figure 9. It can be seen from the figure that the average reward curve of KAI-DDPG reaches a relatively high stable value in about 180000 time steps, while the KA-DDPG needs to reach a relatively high stable value in about 210000 time steps. After calculation, the learning and training speed of KAI-DDPG is nearly 16.67% faster than that of KA-DDPG.

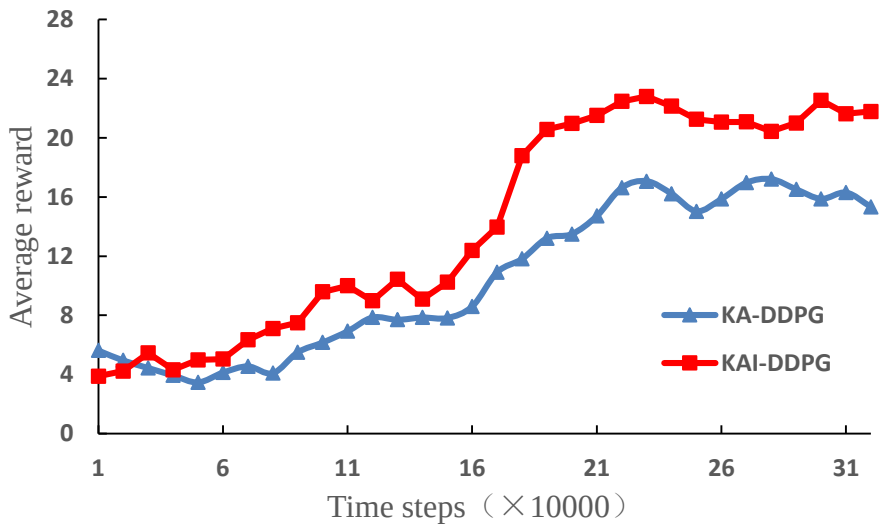


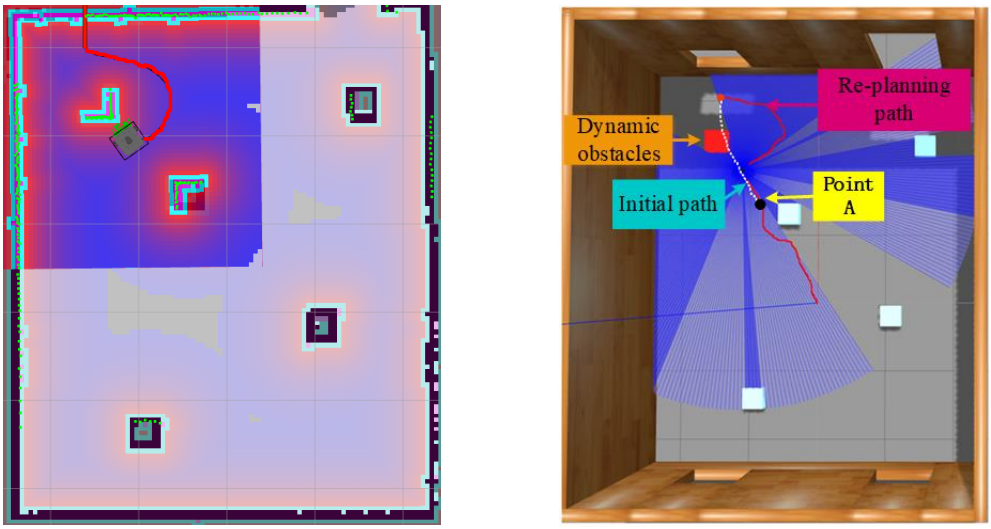
Figure 9 Average reward curve of KA-DDPG and KAI-DDPG in simple simulation scene

Electronic copy available at: https://ssrn.com/abstract=4070518

4.2 Navigation experiment in dynamic simulation scene

4.2.1 Navigation with map based on A* algorithm

As shown in Figure 10, the red line represents the motion path of the mobile robot based on the A* algorithm navigation. Figure 10 (a) shows the view in the Rviz visual interface, and Figure 10 (b) shows the navigation result in the gazebo simulation environment. In which, the path is divided into an initial path and a re-planning path. As shown in Figure 10 (b), the mobile robot first plans the path according to the static environment. When it reaches point a, it suddenly adds red obstacles to simulate the dynamic environment and obtains the re-planning path of the mobile robot. Through experimental verification, it is found that the A* algorithm has a certain ability to deal with dynamic obstacles, and can avoid obstacles and reach the target point.



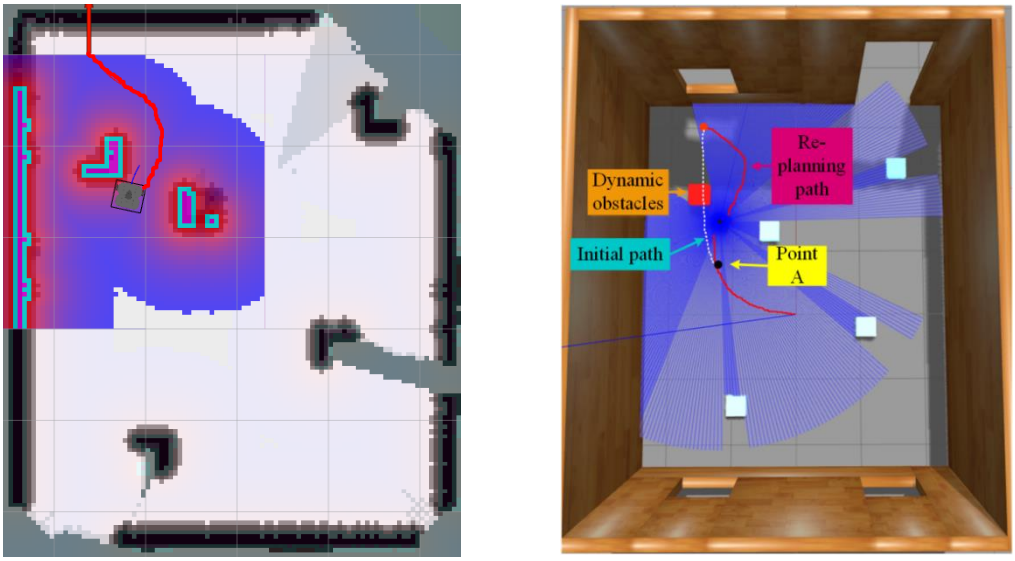
(a)Rviz visual interface (b)Navigation result

Figure 10 Navigation with map based on A* algorithm in dynamic environment

4.2.2 Map-less navigation based on DWA algorithm

Figure 11 (a) shows the Rviz visual interface when the mobile robot navigates with the DWA algorithm, which can help us observe the position relationship between the mobile robot and dynamic obstacles in real time. Figure 11 (b) shows the motion path of the mobile robot based on the DWA algorithm navigation. Through experimental verification, it is found that the mobile robot based on the DWA path planning algorithm has a better ability to deal with dynamic obstacles, and can avoid obstacles and reach the target point with a smoother path.

Electronic copy available at: https://ssrn.com/abstract=4070518

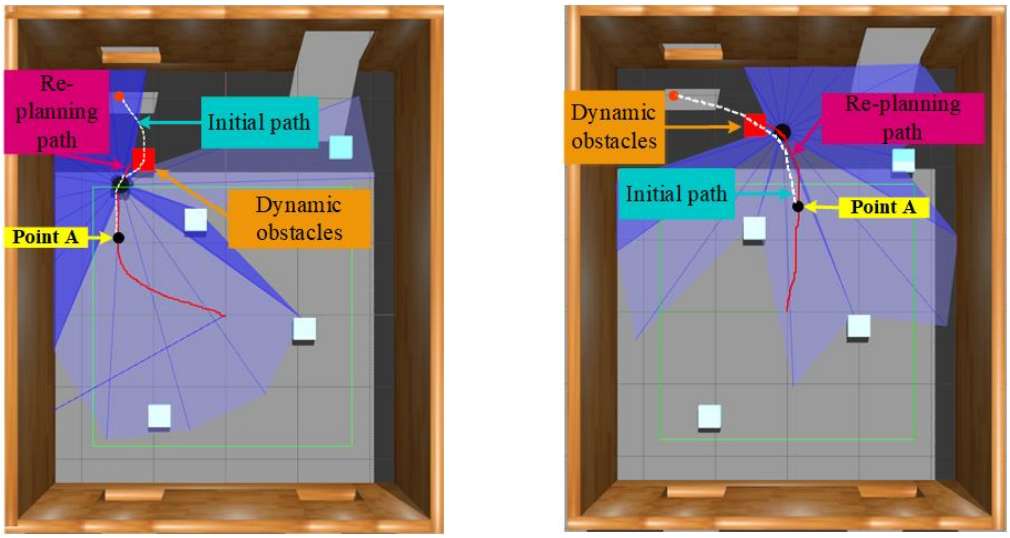


(a)Rviz visual interface (b)Navigation result

Figure 11 Map-less algorithm based on DWA algorithm in dynamic environment

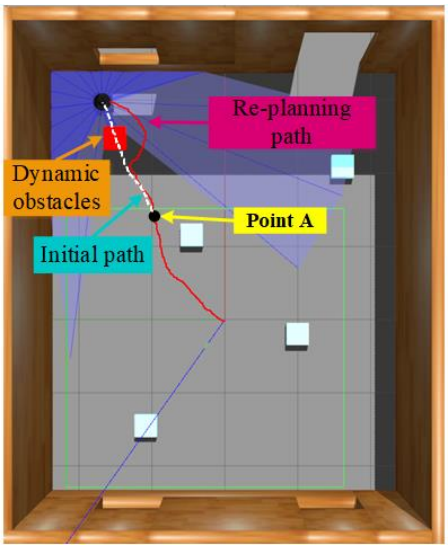
4.2.3 Map-less navigation based on deep reinforcement learning

In the dynamic simulation scene, the motion strategies trained by DQN, DDPG and KA-DDPG in the simple simulation scene are used for map-less path planning. The navigation trajectories based on DQN and DDPG are respectively shown in Figure 12 (a) and Figure 12 (b). It can be seen from the figure that the mobile robot cannot get around the obstacles after introducing dynamic obstacles, which indicates that DQN and DDPG are difficult to adapt to the navigation task in a dynamic environment. The path planning trajectory based on KA-DDPG is shown in Figure 12 (c). It can be seen from the figure that after the introduction of dynamic obstacles, the mobile robot can bypass the obstacles in a smooth path with fast reaction speed and a short navigation path, which indicates that KA-DDPG can better adapt to the dynamic environment and complete various path planning tasks in the dynamic environment.



(a)DQN (b)DDPG

Electronic copy available at: https://ssrn.com/abstract=4070518



(c)KA-DDPG

Figure 12 Map-less navigation based on reinforcement learning in dynamic environment

To sum up, in the dynamic simulation environment, the mobile robot based on KA-DDPG navigation has the strongest adaptability and the best path planning effect; the DWA algorithm is second only to KA-DDPG and has a better ability to deal with dynamic obstacles; although the navigation path of the mobile robot based on A * algorithm is not optimal, it also has a certain ability to deal with dynamic obstacles and can avoid obstacles and reach the target. DDPG and DQN are difficult to adapt to the path planning task in the dynamic simulation environment, and they cannot complete the path planning task smoothly due to collision. The experimental comparison demonstrates that KA-DDPG not only has a good navigation effect, but also some generalization.

4.3 Navigation experiment in complex simulation scene

4.3.1 Comparison with other navigation methods

In the complex simulation scene, the navigation effects of the navigation with map method based on the A* algorithm, the map-less navigation method based on the DWA algorithm and the map-less navigation method based on deep reinforcement learning are compared respectively. The navigation trajectory of each navigation method is shown in Figure 13. It can be seen from the figure that the navigation trajectory based on KA-DDPG is the best, and its trajectory is shorter and smoother than that of other algorithms; the navigation effect of the A* algorithm and DDPG is slightly worse than that of KA-DDPG, but both can make the mobile robot reach the target point smoothly, and the navigation trajectory of the A* algorithm is slightly better than that of DDPG. Although the map-less navigation methods of the DWA algorithm and DQN can also make the mobile robot reach the target point smoothly, their navigation trajectory is poor and takes a long time, and the navigation trajectory of DQN is the worst.

In addition, this experiment records the time required to successfully complete the navigation using each deep reinforcement learning navigation method to prove the effectiveness of KA-DDPG, as shown in Table 4.

Table 4 Navigation time of deep reinforcement learning navigation method

Navigation algorithm	Time consuming(s)
DQN	68
DDPG	45.40
KA-DDPG	38.95

Electronic copy available at: https://ssrn.com/abstract=4070518

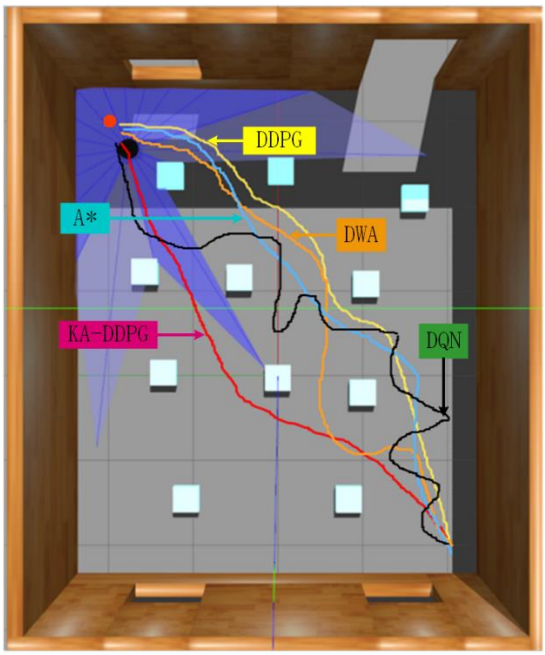


Figure 13 Path planning of each algorithm in complex simulation scene

4.3.2 The success rate of map-less navigation based on DDPG and KA-DDPG

In the training process, the curve of navigation success rate with time step based on DDPG and KA-DDPG is shown in Figure 14. By comparing the navigation success rate curves of DDPG and KA-DDPG, the calculation result shows that DDPG tends to converge in about 210000 time steps, and the average navigation success rate after this is about 76.18%. KA-DDPG tends to converge in about 200000 time steps. After that, the average navigation success rate is about 81.97%, which is 5.79% higher than DDPG.

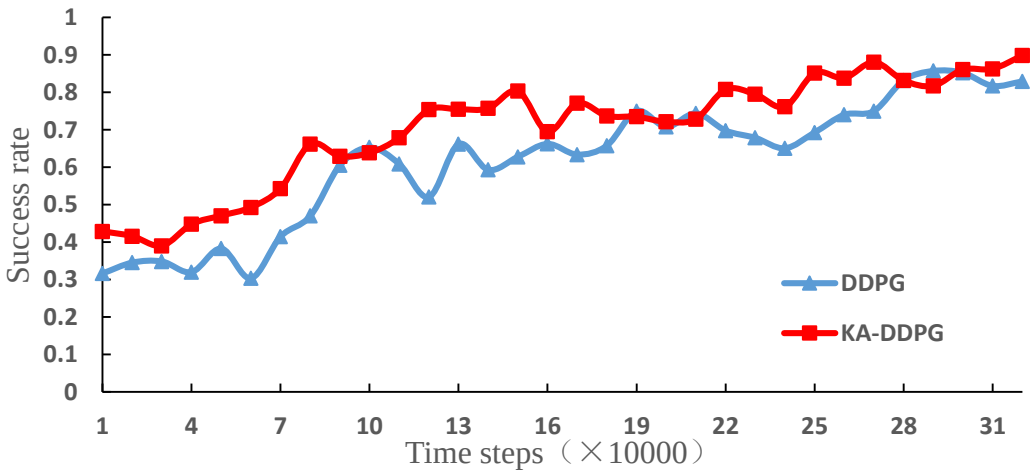


Figure 14 Navigation training success rate of DDPG and KA-DDPG in complex simulation scenes

4.3.3 Training results of map-less navigation based on KA-DDPG and KAI-DDPG

In the training process, the average reward value of KA-DDPG and KAI-DDPG changes with each time step, as shown in Figure 15. It can be seen from the figure that the average reward curve of KAI-DDPG reaches a relatively high stable value in about 200000 time steps, and the curve is smooth with small fluctuations; on the contrary, KA-DDPG needs about 210000 time steps to reach a relatively high stable value. It has been demonstrated that KAI-DDPG is more stable and easier to converge, and that its learning and training speed is nearly 5% faster than that of KA-DDPG.

Electronic copy available at: https://ssrn.com/abstract=4070518

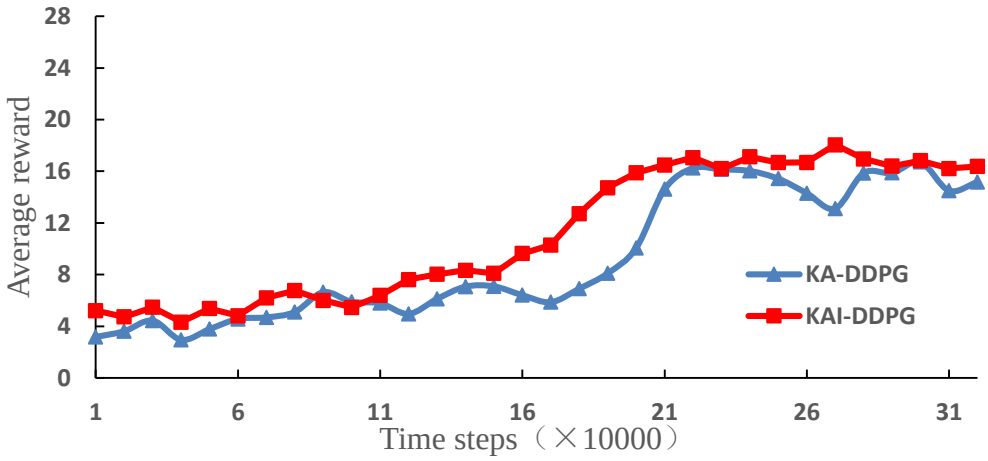


Figure 15 Average reward value of KA-DDPG and KAI-DDPG in complex simulation scenes

5. Practical application

In this paper, KAI-DDPG and DDPG are further applied to three kinds of real environments, which are simple, dynamic and complex. The navigation results are shown in Figures 16-18. The red line in the figures represents the complete navigation path of the mobile robot. From the analysis of the experimental results, it can be concluded that in the real environment, there are problems such as low sensor accuracy and network delay, which lead to the trained model in the simulation environment not being able to adapt to all the real environment. From the figure, KAI-DDPG proposed in this paper can better complete the simple and dynamic real environment navigation tasks, and its navigation performance is greatly improved compared with DDPG. In a complex real environment, although KAI-DDPG and DDPG cannot complete the navigation task, KAI-DDPG navigation task completion is higher than DDPG.

(a)DDPG (b)KAI-DDPG

Figure 16 Comparison of navigation paths between DDPG and KAI-DDPG in simple environment

Electronic copy available at: https://ssrn.com/abstract=4070518

(a)DDPG (b)KAI-DDPG

Figure 17 Comparison of navigation paths between DDPG and KAI-DDPG in dynamic environment

(a)DDPG (b)KAI-DDPG

Figure 18 Comparison of navigation paths between DDPG and KAI-DDPG in complex environment

6. Conclusion

Firstly, by combining the kinematic analysis of mobile robots with the setting of the reward function of DDPG, and introducing the idea of information entropy, KA-DDPG is proposed, and the effect of reward factors of the reward function on path planning is discussed theoretically, which proves the effectiveness of the algorithm. Then, the immune algorithm is used to optimize the experience buffer pool of KA-DDPG, and KAI- DDPG is proposed, which speeds up the training speed of the model; Finally, KAI-DDPG is used to realize the map-less path planning of mobile robots in simple, dynamic and complex simulations and real environments. The experimental results show that compared with DDPG, KA-DDPG can optimize the planning trajectory of a mobile robot, shorten the planning distance and planning time, improve the success rate of path planning, and can better adapt to simple, dynamic and complex simulation scenes; compared with KA-DDPG, KAI-DDPG has a higher utilization efficiency of excellent sample data, which makes the training time shorter.

Acknowledgments

This paper was supported by the Sichuan Science and Technology Program (No. 2020YFG0179)

Electronic copy available at: https://ssrn.com/abstract=4070518

References

[1] Wang, N., Wang, J., & Li, L. (2019). An improved adaptive Monte Carlo localization method for robot. Journal of Navigation and Positioning, 7(03), 31-37. https://doi.org/10.16547/j.cnki.10-1096.20190306.

[2] Zhang, G., Wang, Z., Zhao, Z., Wang, M., & Zhang, X. (2019). Research and Prospect of Mobile Robot Simultaneous Location and Mapping. Software Guide, 18(10), 5-7+22. https://doi.org/10. 11907/rjdk. 191818.

[3] Hart, P. E., Nilsson, N. J., & Raphael, B. (1972). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science & Cybernetics, 4(2), 28-29. https://doi.org/10.1109/TSSC.1968.300136.

[4] Yuan, J., Yang, L., & Tang, X. (2019). Autonomous Vehicle Motion Planning based on Improved RRT* Algorithm and Trajectory Optimization. Acta Automatica Sinica. https://doi.org/10.16383/j.aas.c190607.

[5] Saranrittichai, P., Niparnan, N., & Sudsang, A. (2013). Robust local obstacle avoidance for mobile robot based on Dynamic Window approach. 2013 10th International Conference on Electrical Engineering /Electronics, Computer, Telecommunications and Information Technology.

[6] Roesmann, C., Feiten, W., Woesch, T., Hoffmann, F., & Bertram, T. (2012). Trajectory modification considering dynamic constraints of autonomous robots. Robotics Proceedings of ROBOTIK 2012 7th German Conference on. VDE.

[7] Minsky, Lee, M. (1954). Theory of neural-analog reinforcement systems and its application to the brain-model problem [D]. Princeton: Princeton University.

[8] Bellman, R. A. (1957). Markovian Decision Process. Indiana University Mathematics Journal, 6(4), 15.

[9] Watkins, C. (1989). Learning From Delayed Rewards. Robotics & Autonomous Systems. https://doi.org/10.1016/0921-8890(95)00026-C.

[10] Volodymyr, M., Koray, K., David, S., Andrei, A. R., Joel, V., & Marc, G. B., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533. https://doi.org/10.1038/nature14236.

[11] Hasselt, H. V., Guez, A., & Silver, D. (2015). Deep Reinforcement Learning with Double Q-learning. Computer ence. https://arxiv.org/abs/1509.06461.

[12] Wang, Z., Freitas, N. D., & Lanctot, M. (2015). Dueling Network Architectures for Deep Reinforcement Learning. JMLR.org. https://arxiv.org/abs/1511.06581v1.

[13] Yang, X., Chen, J., Fu, Q., Lu, Y., & Wu, H. (2020). DDPG Based on Stochastic Variance Reduction Method. Computer Engineering and Applications. https://doi.org/10.3778/j.issn.1002-8331.2009-0097.

[14] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., & Tassa, Y., et al. (2015). Continuous control with deep reinforcement learning. Computer ence. https://doi.org/10.1016/S1098-3015(10)67722-4.

[15] Silver, D., Lever, G., Heess, N., Degris, T., & Riedmiller, M. (2014). Deterministic Policy Gradient Algorithms. JMLR.org. https://dl.acm.org/doi/10.5555/3044805.3044850.

[16] Deng, X., Hou, J., Tan, G., Wan, B., & Cao, T. (2020). Multiobjective vehicle following decision algorithm based on reinforcement learning. Control and Decision. https://doi.org/10.13195/j.kzyjc.2020.0426.

[17] Han, J., Ren, H., Wu, S., Jiang, X., & Liu, F. (2020). Robot path planning based on soft AC

Electronic copy available at: https://ssrn.com/abstract=4070518

algorithm for multilayer attention mechanism. Application Research of Computers, 37(12), 3650-3655. https://doi.org/10.19734/j.issn.1001-3695.2019.09.0529.

[18] Zhu, M., Wang, Y., Pu, Z., Hu, J., Wang, X., & Ke, R. (2020). Safe, efficient, and comfortable velocity control based on reinforcement learning for autonomous driving. Transportation Research Part C Emerging Technologies, 117, 102662. https://doi.org/10.1016/j.trc.2020.102662.

[19] Zhang, B., He, M., Chen, X., Wu, C., Liu, B., & Zhou, B. (2019). Self-Driving Via Improved DDPG. Computer Engineering and Applications, 55(10), 269-275. https://doi.org/10.3778/j.issn.1002-8331.1806-0324.

[20] Zhang, H., Xiong, K., & Bai, J. (2019). Improved Deep Deterministic Policy Gradient Algorithm Based on Prioritized Sampling. Proceedings of 2018 Chinese Intelligent Systems Conference, 205-215. https://doi.org/10.1007/978-981-13-2288-4_21.

[21] Shi, S., & Liu, Q. (2019). Deep Deterministic Policy Gradient with Classified Experience Replay. ACTA AUTOMATICA SINICA. https://doi.org/10.16383/j.aas.c190406.

[22] Sun, H., Tong, Y., Tang, S., Tong, M., & Ji, Y. (2018). SLAM Research Based on Kalman Filter and Particle Filter. Software Guide, 17(12), 1-3+7. https://doi.org/10.11907/rjdk.181560.

[23] Zhang, Q., Zhu, M., Zou, L., Li, M., & Zhang, Y. (2020). Learning reward function with matching network for map-less navigation. Sensors, 20(13), E3664. https://doi.org/10.3390/s20133664.

[24] Yonatan H., Kittipong E., Mitsuhiro H., & Waree K. (2020). Reinforcement Q-Learning Control With Reward Shaping Function for Swing Phase Control in a Semi-active Prosthetic Knee. Frontiers in neurorobotics, 14, 565702. https://doi.org/10.3389/fnbot.2020.565702.

[25] Ge, H., & Mao, Z. (2003). Realization of Immune Algorithm. Realization of Immune.

Electronic copy available at: https://ssrn.com/abstract=4070518